Decision Making in Low-Rank Recurrent Neural Networks

Arjun Puri Samuel Delsol Sebastian Valero Dorado
Department of Neuroscience, [Institution]. Correspondence: arjunpur2@gmail.com
July 2026

1 Introduction

1.1 Overview

In this report, we investigate how low-rank RNNs, as an interpretable computational framework, learn to solve decision-making and working memory tasks. Our work builds on Dubreuil et al. 2022, a paper that used the low-rank RNN framework to examine the role of functional subpopulations in solving common cognitive neuroscience tasks. For this report, we scope our work to the learned dynamics of a single population's latents and the statistics of its connectivity parameters.

In essence, the low-rank RNN framework relies on containing the recurrent connectivity of the network to a low-dimensional subspace (via constraining the connectivity matrix), allowing us to both analytically and computationally examine the latent trajectories of the full system in a smaller subspace. We find that the statistics of the recurrent connectivity vectors are sufficient to form an equivalent low-dimensional circuit that can solve the given cognitive tasks with comparable accuracy to the high-dimensional RNN.

In this report, we first lay out the essential ideas of the low-rank RNN framework and how they are used. We examine the findings of rank-one networks on a perceptual decision-making task. Finally, we turn to a parametric working memory task with rank-two networks, contrasting the two results.

1.2 Essence of the Results

2 Methods

In this section, we first discuss the analytical structure of the low rank RNN framework, and then outline the computational method we employed to evaluate the results suggested by the analytical reduction.

In essence, the analytical reduction takes on two steps: we start by explaining how an N-dimensional RNN with low-rank connectivity can be viewed through the lens of a trajectory in a low-dimensional latent space. Then, we reason about how the second-order statistics of the network's loading space (connectivity parameters) give rise to a simple, low-dimensional system that performs the essential computation of the original network. This is where the interpretability power of the low-rank RNN framework lies.

2.1 Model

For our model, we use $N = 128$ leaky neurons driven by recurrent activity and a weighted scalar input. $\mathbf{x}(t) \in \mathbb{R}^N$ is the population state, τ the neural time constant, $\mathbf{I}$ the input weights, $u(t)$ the scalar input, and $z(t)$ a linear readout with weights $\mathbf{w}$:

$$\tau \frac{d\mathbf{x}}{dt} = -\mathbf{x} + \mathbf{J}\phi(\mathbf{x}) + \mathbf{I}u(t),$$
$$\phi(\mathbf{x}) = \tanh(\mathbf{x}), \qquad z(t) = \frac{1}{N}\mathbf{w}^\top \phi(\mathbf{x}).$$

2.2 Latent Dynamics

The core of the low-rank RNN framework rests on constraining $\mathbf{J}$ to rank R via a factorization into left and right connectivity vectors $\mathbf{m}^{(r)}, \mathbf{n}^{(r)} \in \mathbb{R}^N$ for $r = 1, \ldots, R$:

$$\mathbf{J} = \frac{1}{N}\sum_{r=1}^{R} \mathbf{m}^{(r)} \left(\mathbf{n}^{(r)}\right)^\top = \frac{1}{N}\mathbf{M}\mathbf{N}^\top$$

where $\mathbf{M}, \mathbf{N} \in \mathbb{R}^{N\times R}$. For $R = 1$ this reduces to $\mathbf{J} = \frac{1}{N}\mathbf{m}\mathbf{n}^\top$; for the rank-two working-memory networks below, $R = 2$.

In essence, this rank constraint ensures that the population activity $\mathbf{x}(t)$ stays in the low-dimensional subspace spanned by the columns of $\mathbf{M}$ and $\mathbf{I}$. We define the rate projection $\hat{\boldsymbol{\kappa}}(t) = \frac{1}{N}\mathbf{N}^\top \phi(\mathbf{x}(t))$ as the dynamical target for the latent coordinates $\boldsymbol{\kappa}(t) \in \mathbb{R}^R$. Using the basis $\{\mathbf{M}, \mathbf{I}_\perp\}$ to isolate recurrent and input directions, the population state is

$$\mathbf{x}(t) = \mathbf{M}\boldsymbol{\kappa}(t) + \mathbf{I}_\perp v(t).$$

Substituting this decomposition into the RNN equation and matching coefficients along $\mathbf{M}$ and $\mathbf{I}_\perp$ yields the latent dynamics below (see Methods of Dubreuil et al. 2022 - Population structure in computations through neural dynamics (Nature Neuroscience).pdf for details). We are then able to fully describe the low rank network in terms of it's latent dynamics, the input dynamics, and the readout.

$$\tau\dot{\boldsymbol{\kappa}} = -\boldsymbol{\kappa} + \hat{\boldsymbol{\kappa}}, \qquad \hat{\boldsymbol{\kappa}} = \frac{1}{N}\mathbf{N}^\top \phi\big(\mathbf{M}\boldsymbol{\kappa} + \mathbf{I}_\perp v\big),$$
$$\tau\dot{v} = -v + u(t),$$
$$z = \frac{1}{N}\mathbf{w}^\top \phi\big(\mathbf{M}\boldsymbol{\kappa} + \mathbf{I}_\perp v\big).$$

From this, we are able to study the latent trajectories in the basis $\{\mathbf{M}, \mathbf{I}_\perp\}$. Below is a plot showing the coordinates κ, v over the course of the perceptual decision making task. The latent coordinate κ slowly accumulates evidence for the mean stimulus strength as input is provided, and the system converges to a fixed attractor point once the stimulus is turned off, allowing for a linear weighed readout of the result.

2.3 Loading Space

While in the above section we were able to inspect what the low-rank RNNs are doing in the latent space, this section shows how the population as a whole arrives at this computation. The crux of this analytical step is to show that the network of neurons can be described solely by its connectivity parameters in what the original authors term the "Loading Space".

From looking at rate projection, $\hat{\kappa}_r$, we notice that it is built solely from a sum over a function of the network's connectivity parameters. This allows us to treat the rate projection as an empirical average over a distribution of the connectivity parameters where each neuron becomes a point $\mathbf{a}_i = \left(n_i^{(1)}, m_i^{(1)}, \ldots, n_i^{(R)}, m_i^{(R)}, w_i, I_i\right)$ in a $(2R+2)$-dimensional loading space (R dimensions for each n and m, with two added dimensions for w and I).

$$\hat{\kappa}_r(t) = \frac{1}{N}\sum_{j=1}^{N} n_j^{(r)}\,\phi\left(\sum_{r'=1}^{R}\kappa_{r'}(t)\,m_j^{(r')} + I_{\perp,j}\,v(t)\right)$$

In the linear regime of tanh ($\phi(x) \approx x$), this average expands immediately into overlaps of the loading coordinates:

$$\hat{\kappa}_r \approx \sum_{r'=1}^{R} \sigma_{n^{(r)}m^{(r')}}\,\kappa_{r'} + \sigma_{n^{(r)}I_\perp}\,v, \qquad \sigma_{ab} \equiv \frac{1}{N}\sum_{j=1}^{N} a_j b_j,$$

and likewise $z \approx \sum_r \sigma_{wm^{(r)}}\kappa_r + \sigma_{wI_\perp}v$. The latent and readout dynamics are therefore controlled by the covariances of the loading-space cloud. The full nonlinear case replaces these by the same covariances times a population gain (see Methods of Dubreuil et al. 2022 - Population structure in computations through neural dynamics (Nature Neuroscience).pdf for the full explanation). Below, we document the key results without derivation (single population, for clarity). We assume that the parameters come from a zero-mean Gaussian distribution.

$$\tau\dot{\kappa}_r = -\kappa_r + \sum_{r'=1}^{R}\tilde{\sigma}_{n^{(r)}m^{(r')}}\kappa_{r'} + \tilde{\sigma}_{n^{(r)}I_\perp}v,$$

$$z = \sum_{r=1}^{R}\tilde{\sigma}_{wm^{(r)}}\kappa_r + \tilde{\sigma}_{wI_\perp}v,$$

with effective couplings $\tilde{\sigma}_{ab} = \sigma_{ab}\,\langle\Phi'\rangle(\Delta)$, average gain

$$\langle\Phi'\rangle(\Delta) = \frac{1}{\sqrt{2\pi}}\int_{-\infty}^{+\infty} dz\, e^{-z^2/2}\,\phi'(\Delta z),$$

and standard deviation of the recurrent currents

$$\Delta = \sqrt{\sum_{r'=1}^{R}\sigma^2_{m^{(r')}}\kappa^2_{r'} + \sigma^2_{I_\perp}v^2}.$$

For P subpopulations the paper writes more generally $\tilde{\sigma}_{ab} = \sum_{p=1}^{P}\alpha_p\sigma_{ab}^{(p)}\langle\Phi'\rangle_p$ with $\langle\Phi'\rangle_p = \langle\Phi'\rangle(\Delta^{(p)})$

We, however, focus on a single population which results in gain applied across all second-order moments.

Intuitively, this analytical reduction shows that the high dimensional RNN can be viewed as a low dimensional dynamical system governed by the second-order moments of the loading space.

TODO: Tie your results into these equations. Briefly state whether these equations make sense to use in the context of the covariances you identified.

For rank-one and rank-two networks, Dubreuil et al. 2022 - Population structure in computations through neural dynamics (Nature Neuroscience).pdf used the following reduced models. For the perceptual decision-making task (rank one),

$$\begin{aligned}\frac{d\kappa}{dt} &= -\kappa + \tilde{\sigma}_{nm}\kappa + \tilde{\sigma}_{nI}v(t),\\ z &= \tilde{\sigma}_{wm}\kappa + \tilde{\sigma}_{wI}v(t),\\ \tilde{\sigma}_{ab} &= \sigma_{ab}\,\langle\Phi'\rangle(\Delta),\\ \Delta &= \sqrt{\sigma_m^2\kappa^2 + \sigma_I^2 v^2}.\end{aligned}$$

For the parametric working-memory task (rank two), with only the diagonal recurrent covariances kept nonzero,

$$\begin{aligned}\frac{d\kappa_1}{dt} &= -\kappa_1 + \tilde{\sigma}_{n^{(1)}m^{(1)}}\kappa_1 + \tilde{\sigma}_{n^{(1)}I}v(t),\\ \frac{d\kappa_2}{dt} &= -\kappa_2 + \tilde{\sigma}_{n^{(2)}m^{(2)}}\kappa_2 + \tilde{\sigma}_{n^{(2)}I}v(t),\end{aligned}$$

$$\Delta = \sqrt{\sigma^2_{m^{(1)}}\kappa_1^2 + \sigma^2_{m^{(2)}}\kappa_2^2 + \sigma_I^2 v(t)^2}.$$

2.4 Linearized Dynamics and Eigenvalues

- The eigenvalues of our linearized matrix / jacobian show a memory mode (eigenvalue 1) – slow timescale, and a transient mode (small eigenvalue) fast timescale.

2.5 Tasks

Our work focused on two different tasks: 1) perceptual decision making, and 2) parametric working memory.

For the perceptual decision making task, we created input trials by sampling the stimulus strength ($\bar{u}$) from a uniform distribution, then added Gaussian noise ($u(t) = \bar{u} + \xi(t)$). We trained an RNN with a linear readout to predict the dominant evidence (sign of $\bar{u}$) in a noisy signal as a binary decision (+1 or -1). The training objective was the mean squared error between the linear readout and the stimulus strength sign $sign(\bar{u})$.

For the parametric working memory task, we uniformly sampled two values f_1 and f_2 in each trial, and presented these values one after the other with a delay. We trained the network to output the difference between the values. The network needed to learn to encode the earlier value f_1 in it's two dimensional latent space, retrieving during the readout to perform the subtraction. We found that training the network with a fixed delay did not reproduce the dynamics shown by Dubreuil et al. 2022 - Population structure in computations through neural dynamics (Nature Neuroscience).pdf. Instead, the network learned oscillatory dynamics in the latents overfitting to the fixed delay. We changed our training paradigm to use a curriculum learning approach with a variable delay and then reliably reproduced the

line attractor dynamics shown in Dubreuil et al. 2022 - Population structure in computations through neural dynamics (Nature Neuroscience).pdf.

2.6 Training

The networks we used were initialized with 128 neurons and trained over 1000 epochs. To isolate and interpret the recurrent dynamics, we only trained the left and right connectivity vectors m and n, while leaving the other parameters fixed. All parameters were initialized from a standard normal distribution except for w which we sampled with a standard deviation of 4, giving the readout enough range to reach the targets of +1 and -1. Training minimizes a mean-squared error on the readout, masked to the decision period at the end of each trial:

$$\mathcal{L} = \sum_{k,t} M_t \left(z_{k,t} - \hat{z}_{k,t}\right)^2,$$

where k indexes trials, t indexes timesteps, $z_{k,t}$ is the network readout, $\hat{z}_{k,t}$ is the target, and $M_t \in \{0, 1\}$ is nonzero only on decision steps.

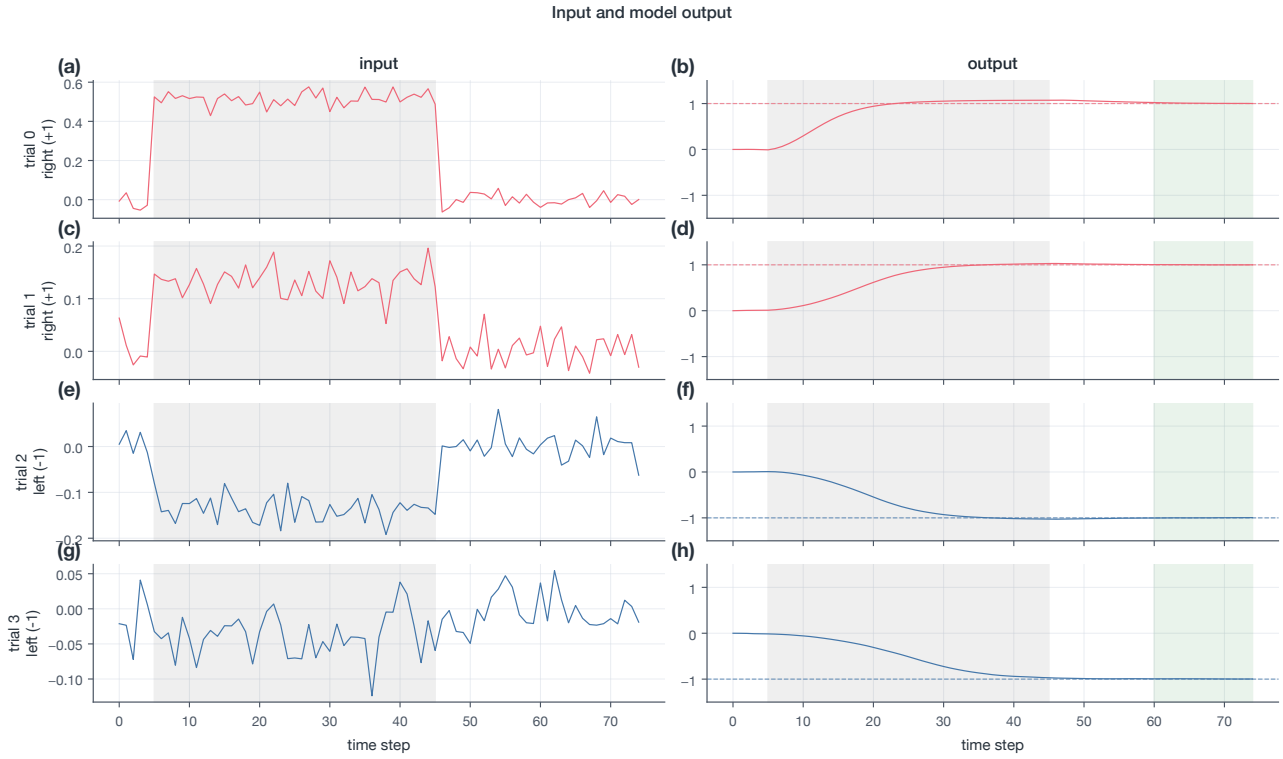

Figure 1. *Representative perceptual decision-making inputs and corresponding rank-one RNN outputs.*

3 Results

In this section, we discuss the computational results found when training rank-one and rank-two RNNs on the perceptual decision-making and parametric working tasks, and how to tie back to the analytical reduction above.

Training rank-one RNNs on the perceptual decision-making task We first look at a rank-one RNN trained on a task to report the mean stimulus strength from noisy evidence (perceptual decision-making). We generated 200 training trials (256 test) where the stimulus strength $\bar{\mu}$ was applied during a fixed time interval. The network was trained to predict the stimulus strength in the last N=15 decision steps of it's output. For this task the recurrent connectivity is constrained to rank one,

$$\tau \frac{d\mathbf{x}}{dt} = -\mathbf{x} + \frac{1}{N}\mathbf{m}\mathbf{n}^\top \phi(\mathbf{x}) + \mathbf{I}u(t),$$

$$\phi(\mathbf{x}) = \tanh(\mathbf{x}), \qquad z(t) = \frac{1}{N}\mathbf{w}^\top \phi(\mathbf{x}).$$

Figure 2. *Four representative trials with strong and weak stimulus strength. Blue represents The stimulus was applied in a subset of the trial, while the network had to predict the stimulus strength in a later time window (the decision window).*

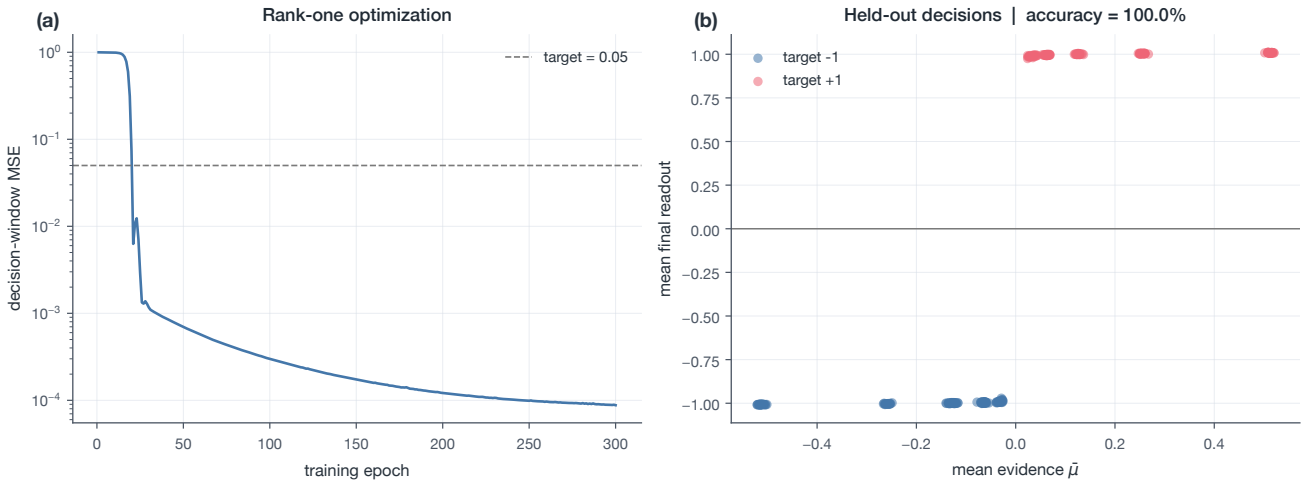

Figure 3. *The mean-squared error converged in under 50 epochs, producing a network that could solve the task with 100% accuracy*

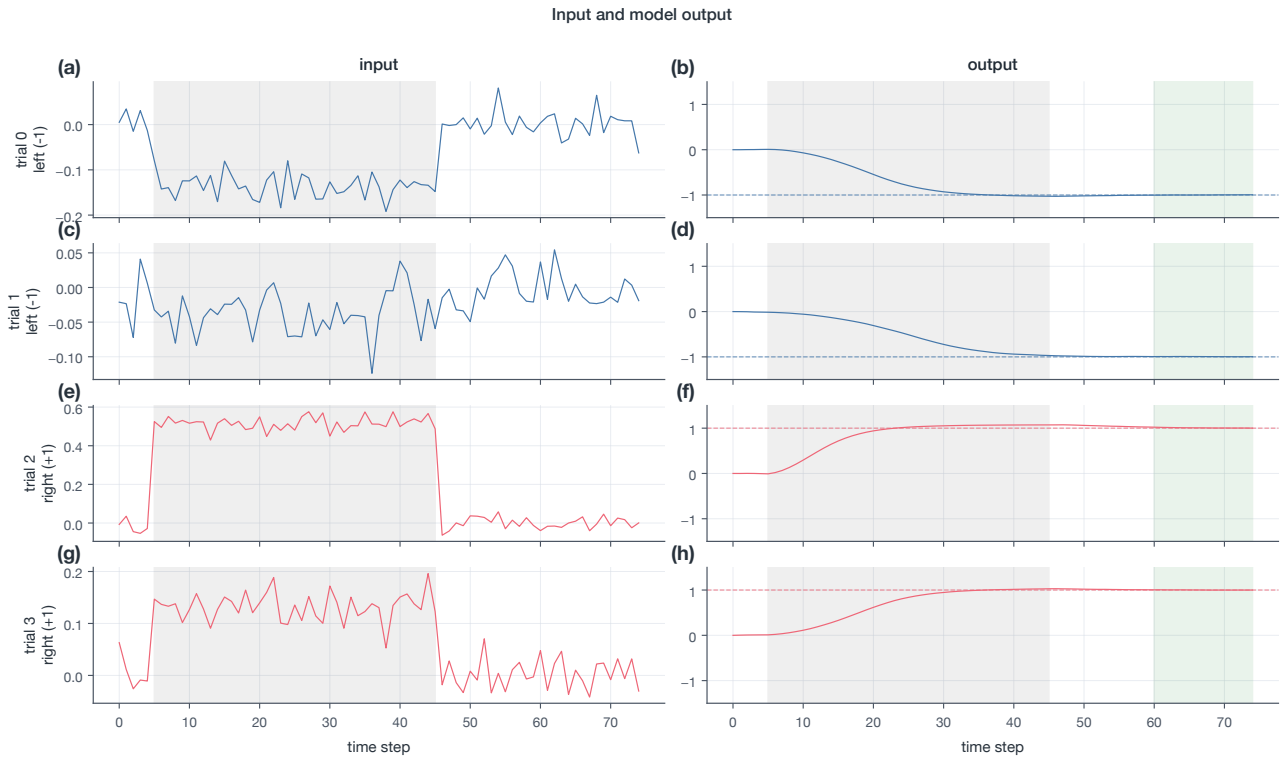

Figure 4. *Model outputs (right) for corresponding inputs (left). The model chooses a direction and sticks with it.*

Interpreting the latent trajectories. With a rank one RNN that successfully performs this decision making task, we now ask how? Earlier, we showed that the population's trajectory lies entirely on the subspace spanned by the left connectivity vector, m and the input mixing vector, I. By projecting the population activity onto an orthogonalized basis that spans this subspace, we observe that the latent trajectories are accumulating evidence and then quickly collapse to a decision along either the positive or negative direction of m depending on the stimulus sign.

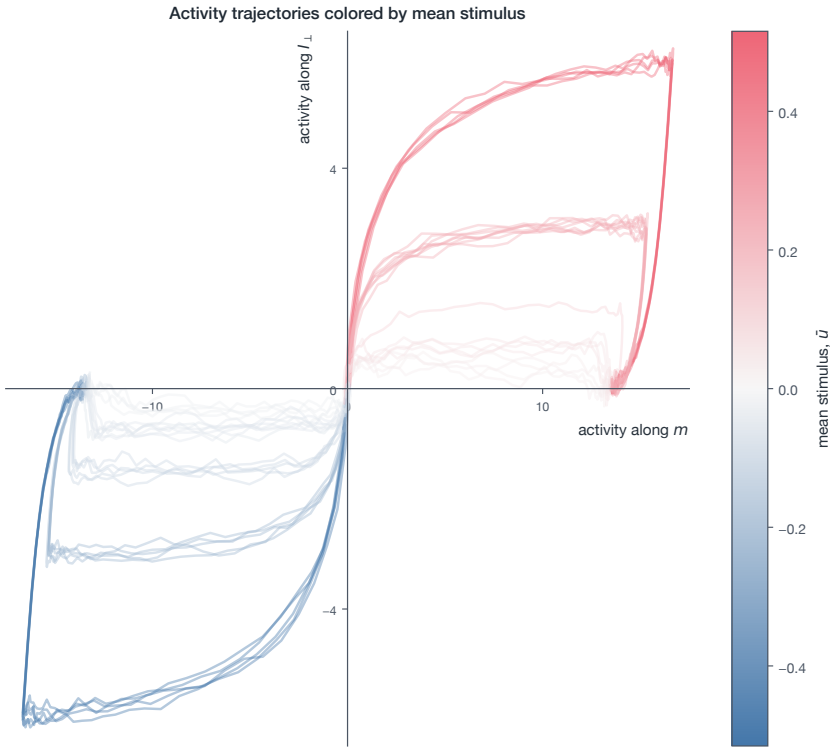

Figure 5. *Test trial population activity, $\mathbf{x}(t) = \mathbf{M}\boldsymbol{\kappa}(t) + \mathbf{I}_\perp v(t)$, projected onto $span(M, I_\perp)$ and graded by trial stimulus strength $\bar{\mu}$ shows excursions from the origin to points on the positive and negative m-axis depending on the sign of the stimulus strength. The network seems to be accumulating signed evidence in latent space, and then collapses to one of the two non-origin fixed points.*

Since the population trajectories lies on a two dimensional subspace, we used PCA on the trajectories to find the directions of maximum variance, and confirmed empircally that most of the variance in the data was explained by the first two components. Projecting the trajectories onto a subspace spanned by these two directions produced a latent trajectory plot similar to what we found above.

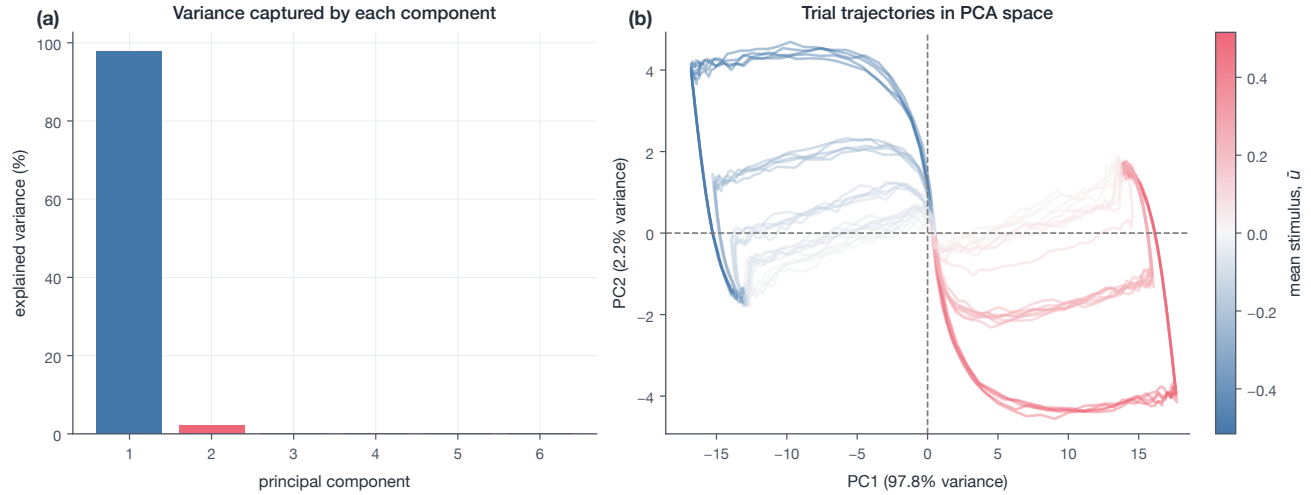

Figure 6. *The first two principal components of the population activity across all trials and time points (right). Projected activity onto the first two principal components reproduces a familiar trajectory to what we found in the basis of m, I, though rotated.*

Covariance between connectivity parameters in loading space. From the previous result we found an interpretable trajectory in latent space for the perceptual decision making task. The network accumulates evidence in latent space and then collapses to one of two directions along m. Now, we ask how the population as a whole achieves this trajectory. To do this, we rely

on the insight from the mean field analysis described above. We saw that the second order moments of the loading space (under the assumption that the parameters are jointly Gaussian) carry the dynamics of the latent. First, we look at the second-order moments of the loading space. **TODO**: How do we interpret these numbers?

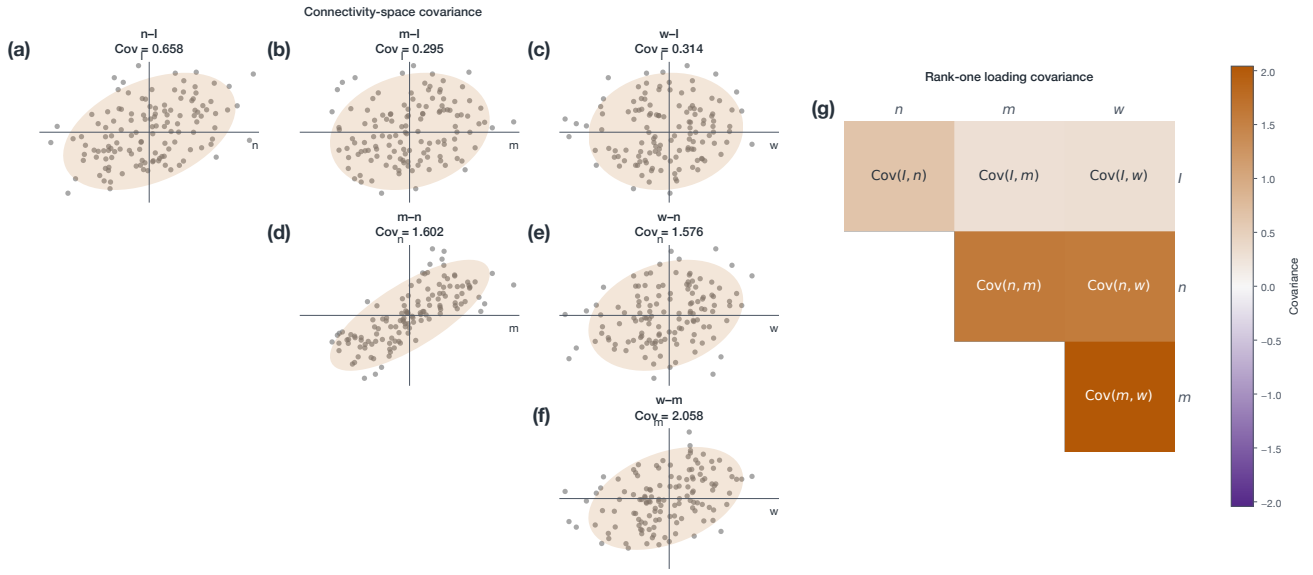

Figure 7. *Plots of the upper triangular covariances of loading parameters. On the left we show the point cloud, and on the right the heatmap. For this particular network run, we found only positive covariances, though in other network runs we had other combinations still resulting in the same predictions*

Re-sampling from the distribution. Next, we show computationally that the distribution of the loading space is sufficient to solve the perceptual decision making task. We fit a four dimensional Gaussian distribution to the point cloud of the trained network's connectivity vectors. From this, we re-construct ten new networks with the same number of neurons as the original network. We test the performance of the sampled networks on the test set and find that they achieve comparable accuracy on predicting the sign of the stimulus strength.

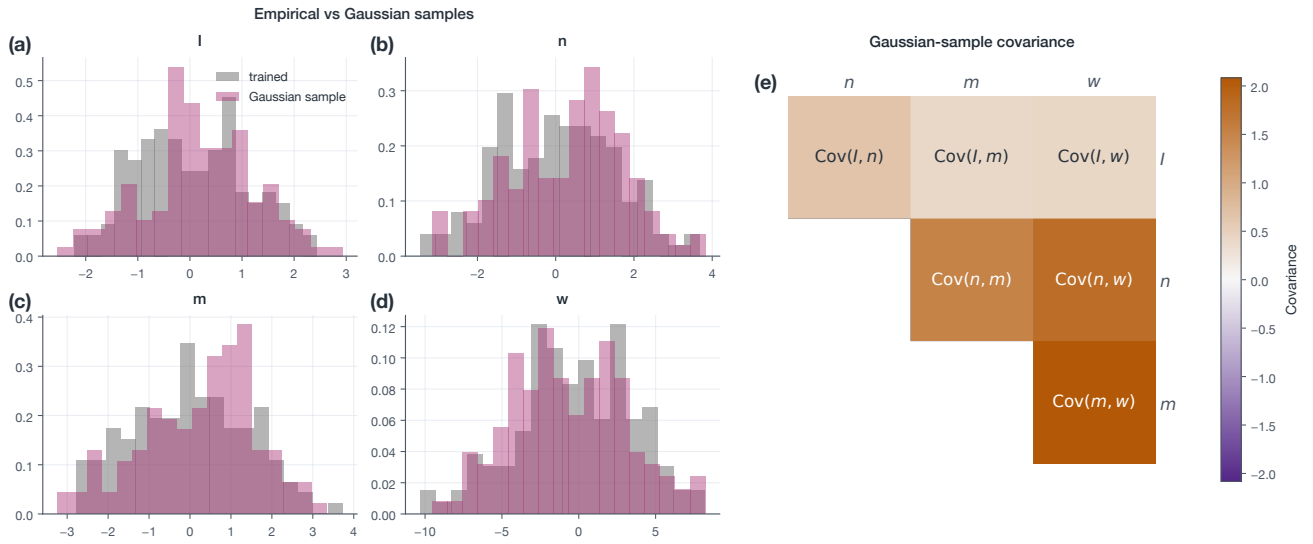

Figure 8. *On the left, we show the distribution of connectivity vectors for a single network. On the right is the covariance heatmap.*

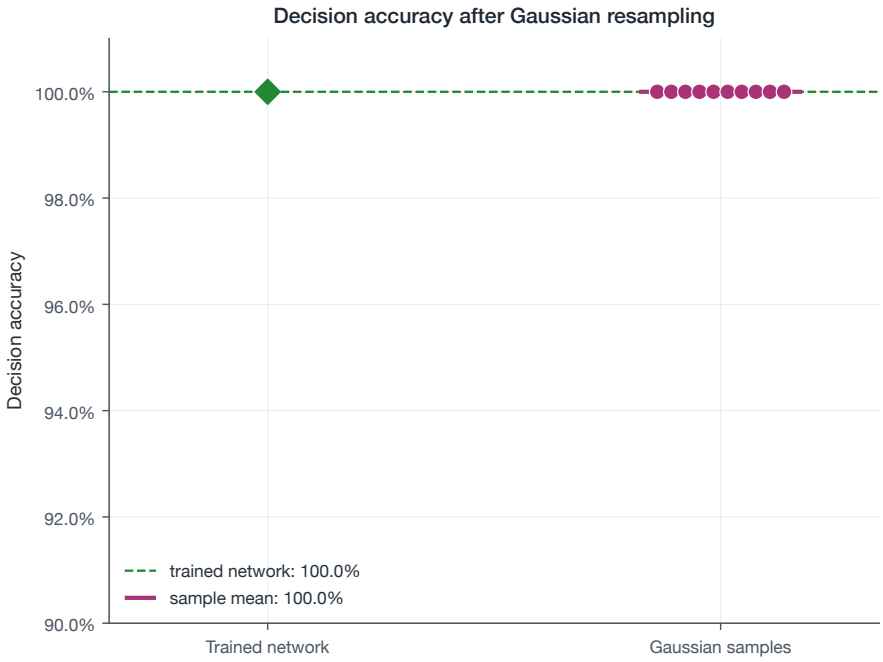

Figure 9. *The Gaussian sampled networks achieve 100% accuracy on the perceptual decision making task, showing that the loading space distribution endows the network with the necessary computation to perform the task*

Equivalent circuit. As discussed above in the methods section, a mean-field analysis of the population dynamics allows us to describe the latent variable κ's dynamics in terms of the covariances of the loading parameters and the average gain of the population.

$$\tau\dot{\kappa} = -\kappa + \tilde{\sigma}_{nm}\kappa + \tilde{\sigma}_{nI_\perp} v = -\big(1-\tilde{\sigma}_{nm}\big)\kappa + \tilde{\sigma}_{nI_\perp} v, \qquad z = \tilde{\sigma}_{wm}\kappa,$$

The recurrent overlap sets an effective timescale $\tau_{\text{eff}} = \tau/(1-\tilde{\sigma}_{nm})$: as $\tilde{\sigma}_{nm} \longrightarrow 1$, integration slows and κ acts as a perfect integrator of the input. Since $\mathbf{w} \perp \mathbf{I}$ by construction, $\sigma_{wI_\perp} = 0$ and the readout depends only on the overlap of $\mathbf{w}$ with $\mathbf{m}$.

$$\tilde{\sigma}_{ab} = \sigma_{ab}\,\langle\Phi'\rangle(\Delta), \qquad \Delta = \sqrt{\sigma_m^2\kappa^2 + \sigma_{I_\perp}^2 v^2},$$

$$\langle\Phi'\rangle(\Delta) = \frac{1}{\sqrt{2\pi}}\int_{-\infty}^{+\infty} dz\, e^{-z^2/2}\,\phi'(\Delta z).$$

The alignment of the latent and recurrent connectivity vectors, $\tilde{\sigma}_{nm}$, changes the timescale of the dynamics, while the alignment of the input and latent vectors, $\tilde{\sigma}_{nI_\perp}$, influences the direction of the driving force of the latent variable κ. $\tilde{\sigma}_{wm}$ gives the network the direction of readout.

We now simulate this one dimensional dynamical system along with the low-pass filtered inputs $\dot{v} = -v + u$, using the covariance parameters of our trained network.

3.1 Rank-two RNN on a parametric working memory task illustrates oscillatory dynamics in latent space. Using curiculum learning with a variable delay produced line attractor dynamics

"For instance we observed that training on the parametric-working memory task with fixed delays between the two stimuli, while they are drawn randomly here, leads to solutions that exploit network oscillations rather than a line attractor" - Dubreuil et al. 2020 - Complementary roles of dimensionality and population structure in neural computations (bioRxiv).pdf

- The paper finds that the latent κ_1 falls on a line attractor that continuously determines f_1.

Speak about the eigenvalues

Speak about the Jacobian / dynamics

3.2 Rank-two RNN on a parametric working memory task illustrates cross-covariances in the non-diagonal terms leading to poor accuracy

4 Discussion